Provably Post-Quantum-Resilient Multi-Agent Llm Orchestration: A Risk-Aware Formal Verification Framework

Abdinasir Ismael Hashi*[1], Aweis Ahmed Hussein[2], Osman Abdullahi Jama[3]

[1, 3] Computer Science, Somali National university, Mogadishu, Somalia.
[2] Computer Science, Capital University of Somalia, Mogadishu, Somalia.

[1]https://orcid.org/0009-0009-0635-2609
[2]https://orcid.org/0009-0005-5425-4377
[3]https://orcid.org/0009-0008-2195-9276
Email: *nasirhaji@snu.edu.so, aweisjarras23@gmail.com, osmanjama@snu.edu.so

Abstract
The rapid adoption of multi-agent Large Language Models (LLMs) creates new security challenges involving compromised agents, prompt injection, privilege escalation, information leakage, communication attacks, and future quantum threats. This study proposes a Provably Post-Quantum-Resilient Multi-Agent LLM Orchestration Framework integrating post-quantum cryptography (PQC), dynamic trust management, risk-aware authorization, secure inter-agent communication, runtime monitoring, and formal verification. Specialized Planner, Reasoning, Retrieval, Security, Verification, and Execution agents are coordinated according to capability, trust, authorization, and contextual risk. Experimental evaluation demonstrates an Attack Detection Rate of 98.72%, Attack Prevention Rate of 97.86%, False Positive Rate of 1.18%, and False Negative Rate of 1.28%. The framework achieved 96.84% task accuracy, 95.72% reasoning accuracy, and 94.63% tool-execution success. PQC operations introduced an average cryptographic overhead of 18.09 ms, while communication latency averaged 14.4 ms. Formal verification achieved 94.1% coverage, with 95% of state transitions automatically verified. These results demonstrate effective security, reliability, adaptability, and quantum resilience for secure deployment of autonomous multi-agent LLM systems in future security-sensitive critical environments.

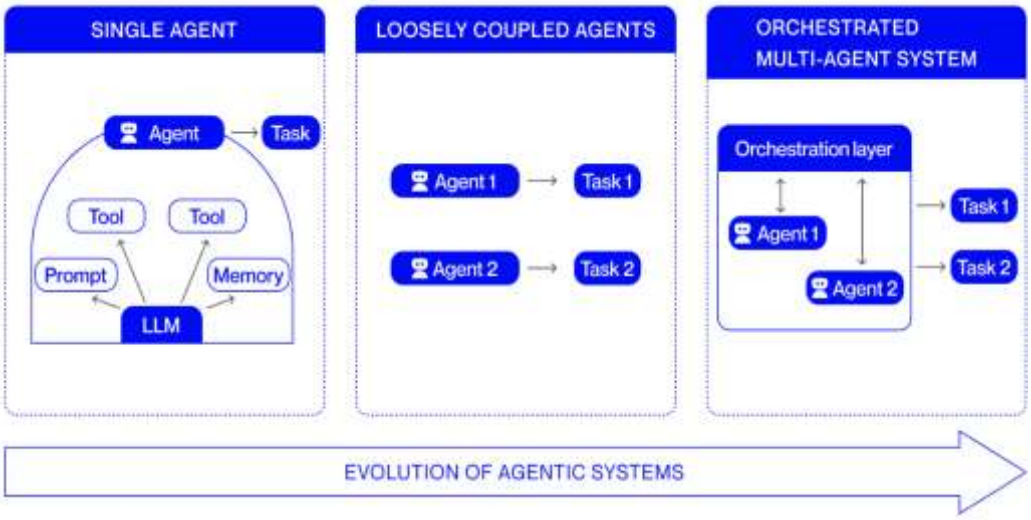
Keywords: Post-Quantum Cryptography (PQC); Multi-Agent LLM; Secure Orchestration; Dynamic Trust Management

1. Introduction

The rapid evolution of Large Language Models (LLMs) has enabled intelligent agents to perform complex reasoning, planning, decision-making, information retrieval, and autonomous task execution across distributed computing environments [1]. Multi-agent LLM orchestration builds on these capabilities by enabling multiple specialized agents to communicate with one another, share context, delegate tasks, and make decisions together to work on a problem as a group [2]. A similar architecture is becoming more significant in Cybersecurity, autonomous systems, healthcare, financial services, cloud computing, and critical infrastructure systems, where specific agents would be able to share their knowledge in a specific manner and an orchestration layer would control the overall process [3-5]. However, the growing autonomy and interconnectivity of LLM agents also introduce significant security and reliability challenges [6]. Some of the potential propagation mechanisms for compromised agents, malicious prompts, unauthorized task delegation, communication interception, identity spoofing, and cascading decision errors include rapid propagation across the agent network [7]. In this context, it has become critical to ensure reliable and trustworthy orchestration mechanisms in order to deploy multi-agent LLM systems in security sensitive contexts [8]. A major emerging concern is the future impact of quantum computing on the cryptographic foundations used to secure agent-to-agent communication and identity

management [9]. Many public-key techniques, including those based on RSA and elliptic-curve cryptography, are based on problems that could be easily solved by powerful enough quantum computers, such as the Shor's algorithm [10]. While there may not yet be many quantum large-scale attackers available, the “harvest now, decrypt later” scenario raises a long-term confidentiality concern for information transmitted and stored today [11]. The threat is even more relevant for multi-agent LLM systems, where agents constantly share credentials, instructions for tasks, context, intermediate reasoning outputs, and sensitive data [12-14]. Thus, the integrity and confidentiality of the entire orchestration process could be compromised in the case that one communication channel is broken. Therefore, in case one communication channel is compromised, the integrity and confidentiality of the entire orchestration process could be compromised [15]. This encourages the adoption of post-quantum cryptography (PQC) in the security structure of multi-agent LLM systems.

Figure 1: Evolution of Agentic Systems [16]

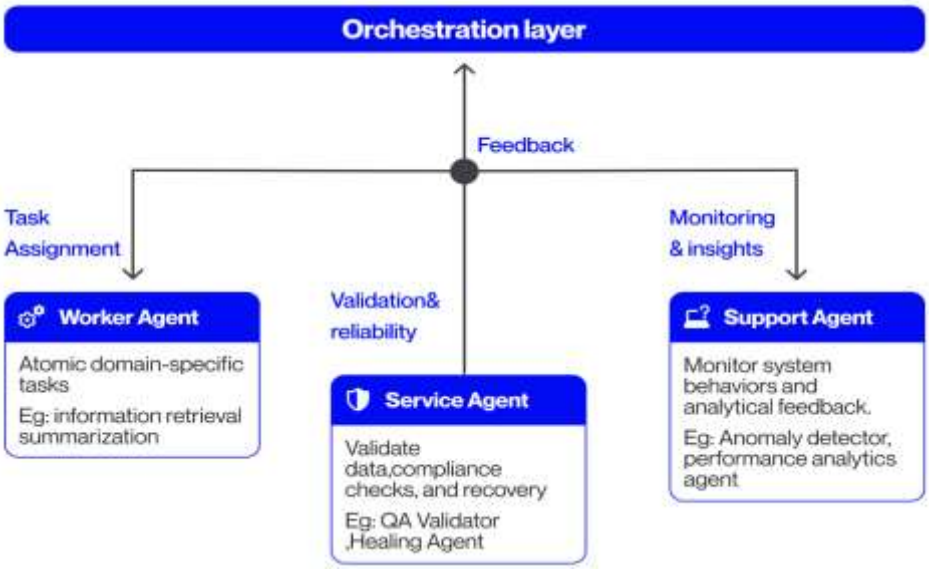


In addition to cryptographic protection, traditional security models are too limited for autonomous agent ecosystems, where security vulnerabilities can arise as a result of the interactions between trusted agents [17]. An LLM agent can generate wrong or distorted output, call an unauthorized tool, take untrusted input or choose an inappropriate agent for a task [18]. These risks are greater in the context of dynamic agents, and decisions generated by probabilistic language-model behavior [19]. Thus, security cannot rely solely on encryption and authentication, but it should also include periodic risk analysis, trust analysis, policy verification, behavior tracking and formal verification [20]. A risk-aware orchestration framework needs to assess the security condition of agents and communications before granting access to sensitive operations, and dynamically flex security permissions based on observed behaviour, contextual risk and security policies [21].

Formal verification gives a systematic way to tackle this requirement by mathematically defining system properties and verifying whether an architecture meets a set of security and operational requirements [22]. In the context proposed here, formal verification may be used to establish properties of authentication, confidentiality, integrity, authorization, secure delegation, communication consistency and fault containment [23]. The framework can also model agent interactions in terms of formally-defined states, transitions, policies, and security invariants, instead of just empirical tests. In the case of

a failure, the orchestration mechanism will be able to pinpoint the failure's corresponding transition or interaction [24]. This gives greater confidence in multi-agent environments that are risky because if specific execution scenarios are limited, then there is a risk that there are critical vulnerabilities that have not been detected.

Figure 2: Specialized agents in a Multi-Agent System [16].



The proposed research, thus, proposes a Provably Post-Quantum-Resilient Multi-Agent LLM Orchestration: A Risk-Aware Formal Verification Framework, which combines the post-quantum cryptographic protection, decentralized agent identity, trust management, risk-aware access control, secure communication, and formal verification into a single framework. All participating agents have a unique identity and are subjected to specified security policies and capabilities. PQC mechanisms are introduced to enhance the authentication, key establishment, and secure communication from the expected quantum attacks. Factors of agent behaviour, communication anomalies, task sensitivity, trust levels, and exposure to attack are continuously assessed to determine the risk. The orchestration layer can then control access to task delegation, communication and execution permissions based on the resulting risk score. Her are the research objectives of the study follows as:

- Implement secure orchestration among specialized LLM agents.
- Model LLM-specific, communication, privilege, information-leakage, and quantum-enabled threats.
- Integrate a post-quantum cryptographic security layer for agent identity and protected communication.
- Continuously update agent trust using security and behavioral evidence.
- Apply risk-aware authorization before sensitive and high-impact operations.
- Formally evaluate authentication, authorization, confidentiality, workflow integrity, and post-quantum security properties.
- Evaluate Security, Cryptography, Communication, AI Performance, Verification, Risk, Reliability, and Scalability.

2. Related Work

Intelligent reasoning, collaboration capabilities and tool-based decision-making are the major advantages of autonomous LLM agents, but they also present great challenges in terms of security, trust, authorization, communication, and verification. Yao et al., 2022

[25] proposed their reasoning-and-action generation framework ReAct, where reasoning and action are generated to interact with external environments, but they did not consider secure inter-agent communication, cryptographic identity, quantum resistance, or formal security guarantees. While AutoGen [26] is an important step in customizable LLM agents that can interact and exchange information with humans and external tools, it doesn't solve the problems of post-quantum authentication, dynamic trust evaluation, and risk-based privilege adaptation. Greshake et al. (2023) [27] highlighted the dangers posed by IPI, presenting a method for exploiting external data to cause the LLM to behave in ways that are unintended. Wallace et al. (2024) [28] focused on privilege differentiation using instruction hierarchy, and Huang et al. (2024) [29] highlighted the importance of verifying, validating, monitoring at runtime, and formalizing trustworthy LLM systems. Overall, these studies confirm that it is important to secure the execution of agents, to control their privileges, and to formally verify their trustworthiness, but most of these works consider only a single agent or a single type of security violation rather than provide full coverage for distributed multi-agent workflows.

Recent studies have also reinforced the security aspect by analyzing the capabilities of agents, assessing risk, and exploring post-quantum cryptography. AgentDojo, released by Debenedetti et al. in 2024 [30] contained 97 realistic tasks and 629 security test cases to test prompt-injection attacks and defenses in tool using agents, while Agent Security Bench (ASB) was presented by Zhang et al. in 2025 [31] with 10 scenarios, 10 agents, over 400 tools, 23 attack/defense methods and 8 evaluation metrics. Alquwayfili et al. (2025) [32] came up with Quantigence, which integrated multi-agent orchestration, cryptographic analysis, threat modelling, standards compliance and risk assessment, with tool-supported processing bringing the accuracy up to 98% and the complex-query rubric coverage to 89%. Drammeh et al. (2025) [33] showed that multi-agent orchestration is effective for incident response, with a high actionable recommendation rate (100% vs. 1.7% for single-agent systems). Specifically, Campbell et al. (2026) [34] focused on migration of PQC for agentic AI and pointed out the issues of confidentiality and integrity that may arise with future quantum attacks, highlighting the migration of critical trust anchors. Although these advances have been made, there is significant scatter and overlap in the literature on orchestration, attack detection, benchmarking, risk assessment, formal verification, and PQC migration. A clear gap therefore exists in integrating post-quantum-resilient identity and communication, dynamic risk-aware trust and authorization, continuous security monitoring, and formal verification within a single multi-agent LLM orchestration framework.

3. Research Methodology

3.1 System Architecture and Agent Definition

The proposed Provably Post-Quantum-Resilient Multi-Agent LLM Orchestration Framework will be layered, incorporating a set of specialized LLM agents, post-quantum cryptography (PQC), dynamic trust management, risk assessment, formal verification, and runtime security monitoring. The framework responds to the user's request by converting it into a structured task representation $T = \{ID, G, C, P, S\}$ with ID a task identifier, G a goal, C a set of constraints, P a task priority and S a task sensitivity. The orchestration controller breaks down the task into subtasks $P_T = \{S_1, S_2, \ldots, S_n\}$ and dynamically allocates them to suitable agents based on their capabilities, authorization levels, trust scores and related security risks.

Table 1: Multi agent layer

Agent	Primary Role	Initial Trust	Authorization
AG-001	Planner Agent	0.91	3
AG-002	Reasoning Agent	0.89	3
AG-003	Retrieval Agent	0.87	2
AG-004	Security Agent	0.94	4
AG-005	Verification Agent	0.93	4
AG-006	Execution Agent	0.90	3

The multi-agent layer includes Planner Agent, Reasoning Agent, Retrieval Agent, Security Agent, Verification Agent and Execution Agent. The Planner Agent will be used to determine the best sequence of operations $P_T = f_{plan}(T)$, and the Reasoning Agent will be used to generate intermediate solutions according to $O_R = f_{LLM}(S_i, C_i)$. The Retrieval Agent has access only to information sources which have been authorized while the Security Agent identifies malicious instructions, anomalous activities, privilege violations, and possibly unsafe requests. The Verification Agent checks the output generated by the algorithm against a set of policies such as $V(O_i) = 1$ when $O_i| = Policy$. The Execution Agent is permitted to invoke external tools or APIs only when all security conditions are satisfied:

$$Auth(A_i) \Lambda Authorized(A_i, o) \Lambda Trust(A_i) \geq T_{min} \Lambda Verify(o) = 1$$

Agent communication is represented as a directed graph $G = (V, E)$ with vertices representing agents, and edges representing communication channels authorized by the agents. An agent has an identity $ID_i = \{PK_i, SK_i, Cert_i\}$ and communication with other agents is secured by PQC key establishment and digital signatures. When sending a message M, the sender computes $\sigma_i = Sign(sk_i, M)$, which is accepted only when $Verify(pk_i, M, \sigma_i) = 1$.

Dynamic trust is continuously updated using

$$T_i = \alpha A_i + \beta B_i + \gamma V_i + \delta H_i$$

A_i, B_i, V_i, and H_i are the authentication status, behavioural reliability, verification history, and security performance, respectively. Runtime risk is quantified as $R_i = P_i \times I_i \times E_i$, representing probability, impact, and exposure. Agents with $R_i > R_{critical}$ will be automatically restricted or quarantined. Finally, formal verification is used to establish security invariants like $\square(\neg UnauthorizedAccess)$ and $\square(CriticalAction \rightarrow VerifiedAction)$. Therefore, in order to be able to carry out the critical operations, cryptographic authentication and authorization checks have to be complemented by a process of trust evaluation, risk assessment and formal verification that has established, together, that the requested operation complies with the security requirements of the framework.

3.2 Threat and Risk Modeling

The proposed Provably Post-Quantum-Resilient Multi-Agent LLM Orchestration Framework features an extensive threat and risk modeling process to detect, measure, prioritize, and dynamically manage security threats during multi-agent communication. The proposed framework not only tackles network and software vulnerabilities but also considers threats specific to LLMs such as malicious and compromised agents, attacks on inter-agent communication, sensitive data leakage, privilege escalation, prompt injection, retrieval poisoning, and threats arising from post-quantum cryptography. Threat space T

is represented as $\mathcal{T} = \{T_1, T_2, \ldots, T_n\}$ where each T_i represents a threat that has been identified to an agent, asset, communication channel or execution service.

Table 2: Threat with their severity

Threat	Severity
Prompt Injection	0.72
Compromised Agent	0.88
Impersonation	0.84
Replay Attack	0.70
Message Manipulation	0.76
Privilege Escalation	0.91
Retrieval Poisoning	0.79
Sensitive Information Leakage	0.86
Malicious Tool Invocation	0.94
Denial of Service	0.67
Quantum-Enabled Cryptographic Attack	0.97

The framework identifies, for each of the threats identified, the asset that will be affected, how they will affect it, the likelihood of the threat occurring, the potential impact if it does, the level of exposure, and what security control is applicable to the threat. As such, the threat can be represented as a set of attributes, namely $T_i = \{A_i, P_i, V_i, I_i, E_i, C_i\}$, where A_i is the affected asset, P_i is the probability of the threat occurring, V_i is the attack vector, I_i is the potential impact, E_i is the exposure, and C_i is the security control. Major threat categories include direct and indirect prompt injection, compromised agents, agent impersonation, replay attacks, message manipulation, unauthorized privilege escalation, poisoned retrieval data, sensitive-information leakage, malicious tool invocation, denial-of-service behavior, and quantum-enabled cryptanalytic attacks.

A quantitative risk score is calculated as:

$$R_i = P_i \times I_i \times E_i$$

where all parameters are normalized between 0 and 1. The overall system risk is determined using:

$$R_{total} = \sum_{i=1}^{n} w_i R_i$$

where w_i is the threat's criticality weight assigned to threat T_i where $\sum w_i = 1$. This weighted formulation allows the framework to rank threats based on their impact and significance to the sensitive operations.

The framework additionally determines agent-specific risk using:

$$R(A_i) = \alpha R_B + \beta R_C + \gamma R_P + \delta R_E$$

where R_B, R_C, R_P, and R_E are the risks of behavior, communication, privilege and execution, respectively. A higher risk score occurs when an agent behaves in an unusual manner, attempts unauthorized access, breaches communication policies or repeatedly fails verification. The orchestration controller then lowers the privileges, raises the verification processes or quarantines the agent.

The framework also considers the potential for attack propagation between agents as the agents communicate through a directed graph, $G = (V, E)$. The propagation risk from agent A_i to agent A_j is expressed as:

$$R_{ij} = R_i \times P_{ij} \times I_j$$

where P_{ij} denotes the probability that a compromise propagates from A_i to A_j, while I_j represents the impact on the receiving agent. Communication paths with a higher propagation risk can thus be isolated, rate limited or put under further authentication or verification.

Post-quantum cryptographic exposure is separately incorporated through:

$$R_{PQC} = P_{break} \times I_{crypto} \times E_{crypto}$$

where P_{break} is the probability of cryptographic compromise, I_{crypto} is the impact of cryptographic compromise and E_{crypto} is the cryptographic exposure. This formulation ensures that any deficiencies in authentication, key establishment, digital signatures or protected communication are factored into the overall risk assessment.

Based on the calculated risk, each operation is categorized as Low, Moderate, High, or Critical. The corresponding decision mechanism is:

$$Decision = \begin{cases} Allow, & R < R_L \\ Monitor, & R_L \le R < R_M \\ Verify, & R_M \le R < R_H \\ \dfrac{Block}{Quarantine}, & R \ge R_H \end{cases}$$

Risk is continuously updated during system execution according to:

$$R_t = f(R_{t-1}, A_t, C_t, B_t, O_t)$$

A_t, C_t, B_t, safand O_t are attack events, cryptographic events, behavioral events and operational events observed at time t, respectively. Thus, the proposed framework combines static threat identification, quantitative risk scoring, agent-specific risk evaluation, analysis of how threats propagate in an attack, post-quantum security assessment and runtime monitoring. This allows the orchestration controller to dynamically adjust security policies, limit the access of compromised agents, isolate risky communication paths and block high-risk operations from accessing sensitive resources.

3.3 Post-Quantum Cryptographic Security Layer

The proposed structure includes the integration of a PQC Security Layer to ensure the security of communication, authentication, session setup, and data transfer between collaborating LLM agents. The framework proposes a set of quantum-resistant cryptographic mechanisms to be added to the agent communication pipeline in order to make conventional public-key cryptographic mechanisms less vulnerable to the possibility of a sufficiently capable quantum computer. Each agent is assigned a unique cryptographic identity represented by $ID_i = \{PK_i, SK_i, Cert_i\}$, where PK_i denotes the public key, SK_i represents the private key, and $Cert_i$ is the corresponding digital authentication credential. An agent must be registered in the trust and authorization system before it is allowed to participate in orchestration.

Once authenticated, communicating agents set up a protected session with a PQC key establishment mechanism. The common session key could be represented as:

$$K_{ij} = Decaps(sk_j, CT_{ij})$$

In this case, CT_{ij} represents the ciphertext encapsulated in the key-establishment procedure. If one of these two agents is compromised, then the other agent can use its session key to ensure that the communication between the two is protected by authenticated encryption. This means that important information for a task, intermediate reasoning results, data retrieved, authorization information, and execution commands are not exposed in the process of being transmitted.

Message authenticity and integrity are further maintained through PQC digital signatures. When agent transmits a message M_{ij} to agent A_j, the sender generates:

$$\sigma_i = Sign(sk_i, M_{ij})$$

The receiving agent validates the message using:

$$Verify(pk_i, M_{ij}, \sigma_i) = 1$$

Communication is accepted only if the sender identity is verified, and a secure communication session is successfully established, and the digital signature is successfully verified. As a whole, the condition of communication security can then be stated as:

$$Secure(M_{ij}) = Auth(A_i) \wedge PQC(K_{ij}) \wedge Verify(\sigma_i) = 1$$

If there are any conditions that are not met, the message is rejected or passed on to the security and verification layers to be examined. The mechanism helps mitigate the following threats: agent impersonation, message modification, replay, unauthorized communication, session interception, and cryptographic compromise. It also supports a harvest-now-decrypt-later threat model by protecting sensitive information with cryptographic mechanisms designed to remain secure against anticipated quantum capabilities.

The Trust and Identity Management Layer ensures continuous identity verification of agents and dynamically assesses the trustworthiness and security level of the participating agents. Identity verification happens when an agent first logs onto the orchestration environment, and when the agent needs to access resources that require sensitive privileges, when the agent accesses resources that require protection, when the agent communicates across a trust boundary, or when the agent performs high-impact operations. Notably, successful authentication is in itself not enough to prove trustworthiness. The proposed framework is thus an integration of cryptographic identity and behavioral attributes, authorization status, verification history, and the previous security performance.

The trust value of agent A_i at time t is calculated as:

$$T_i(t) = \alpha A_i(t) + \beta B_i(t) + \gamma V_i(t) + \delta H_i(t)$$

$A_i(t)$ is the level of confidence in authentication, $B_i(t)$ is the level of reliability in behavior, $V_i(t)$ is the degree of compliance in verification, $H_i(t)$ is the historical security performance. The weighting parameters meet the following criterion:

$$\alpha + \beta + \gamma + \delta = 1$$

The trust value is continuously updated as the agent performs tasks, exchanges messages, accesses resources, requests additional privileges, or encounters security violations. This assessment is a dynamic assessment that allows the framework to differentiate between the behavior of the agent that is authenticated and trusted and the behavior of the agent that is authenticated but which shows indications of having been compromised in recent activities.

Agent permissions are dynamically determined according to both trust and authorization policies. The access condition can be:

$$Access(A_i, o) = 1$$

If

$$Auth(A_i) \Lambda T_i \geq T_{min} \Lambda Policy(A_i, o) = 1$$

where represents the requested operation and is the minimum trust threshold. High trust agents may have access to normal privileges for tasks while moderate trust agents may have to perform extra verification or access low-risk tasks. Low Trust Agents may be sheltered, quarantined, or isolated until their security status can be reevaluated.

Trust degradation happens when an agent exhibits malicious behavior, is found to be in violation of policies, is communicating in an unusual manner, or has not been cryptographically verified. The simplified trust-update mechanism is defined by:

$$T_i(t+1) = T_i(t) - \lambda V_i$$

where V_i represents the severity of the detected security violation and denotes the penalty coefficient. Conversely, successful completion of verified operations can contribute to

gradual trust restoration. This adaptive mechanism prevents a compromised or misbehaving agent from retaining excessive privileges after its security condition changes.

Table 3: Risk-Aware Authorization Decisions for Multi-Agent Operations

Request	Agent	Operation	Risk	Decision
AUTH-001	AG-001	Task Planning	0.18	ALLOW
AUTH-002	AG-002	Generate Intermediate Reasoning	0.32	REQUIRE_VERIFICATION
AUTH-003	AG-003	Retrieve Sensitive Information	0.35	REQUIRE_VERIFICATION
AUTH-004	AG-004	Security Monitoring	0.36	REQUIRE_VERIFICATION
AUTH-005	AG-005	Formal Policy Verification	0.24	ALLOW
AUTH-006	AG-006	Controlled Tool Execution	0.48	REQUIRE_VERIFICATION
AUTH-007	AG-003	Privileged Tool Execution	0.62	BLOCK
AUTH-008	AG-006	High-Risk External Operation	0.71	RESTRICT

The integration of PQC security with dynamic trust management produces a cryptographically authenticated and continuously verified multi-agent ecosystem. PQC mechanisms provide confidence in the identity of the agent, in the establishment of a session, in the authenticity of the messages sent/received, and in the integrity of the communication, while trust management provides the basis for deciding whether an authenticated agent should be granted further privileges. The framework is therefore based on the basic idea that authentication provides identification of the agent, and dynamic trust provides authorization for the agent. This integrated approach enhances the protection against quantum-based attacks, compromised agents, privilege escalation and unauthorised use of resources while also offering the formal basis for verifying identities, authorisations, communication and privilege management properties.

3.4 Formal Modeling of Agent Orchestration

The orchestration protocol is represented using a formal state-transition model. A system state is defined as:

$$S = (A, T, P, K, R, O)$$

where A denotes agent states, T represents trust values, P represents permissions, K represents cryptographic session states, R represents risk states, and O represents task outputs.

Table 4: Formal Security Property Evaluation

Formal Property	Evaluation
Authentication	Verified
Authorization	Verified
Confidentiality	Verified
Workflow Integrity	Verified

Post-Quantum Security	Verified
Replay Protection	Verified
Runtime Safety	Verified

Agent transitions are represented as:

$$S_t \xrightarrow{a} S_{t+1}$$

where is an authorized action. Formal specifications are developed for critical properties such as:

Authentication property: $Authenticated(A_i, A_j) \Rightarrow ValidSignature(M)$
Authorization property: $Execute(A_i, o) \Rightarrow Authorized(A_i, o)$
Confidentiality property: $Sensitive(M) \Rightarrow Encrypted(M)$
Workflow integrity property: $CriticalAction \Rightarrow Verified(Action)$
Post-quantum security property: $Communication(A_i, A_j) \Rightarrow PQCProtected(A_i, A_j)$

These properties provide machine-checkable conditions rather than relying exclusively on empirical testing.

3.5 Performance Evaluation

Security improvements are evaluated together with computational overhead. The principal evaluation metrics are:

Table 5: Performance Evaluation

Category	**Metrics**
Security	Attack Detection Rate, Attack Prevention Rate, False Positive Rate, False Negative Rate
Cryptography	Key-Establishment Time, Signature Generation Time, Verification Time
Communication	Latency, Throughput, Message Overhead
AI Performance	Task Accuracy, Reasoning Accuracy, Tool-Execution Success
Verification	Property Coverage, Number of Verified Properties, Counterexamples
Risk	Mean Risk Score, Critical-Risk Detection Rate
Reliability	Task Completion Rate, Agent Failure Recovery Rate
Scalability	CPU Usage, Memory Usage, Agent Scaling Overhead

The cryptographic overhead is calculated as:

$$O_{crypto} = \frac{T_{PQC} - T_{baseline}}{T_{Baseline}} \times 100$$

while security improvement is calculated as:

$$SI = \frac{D_{proposed} - D_{baseline}}{D_{baseline}} \times 100$$

where D represents the relevant security-performance metric.

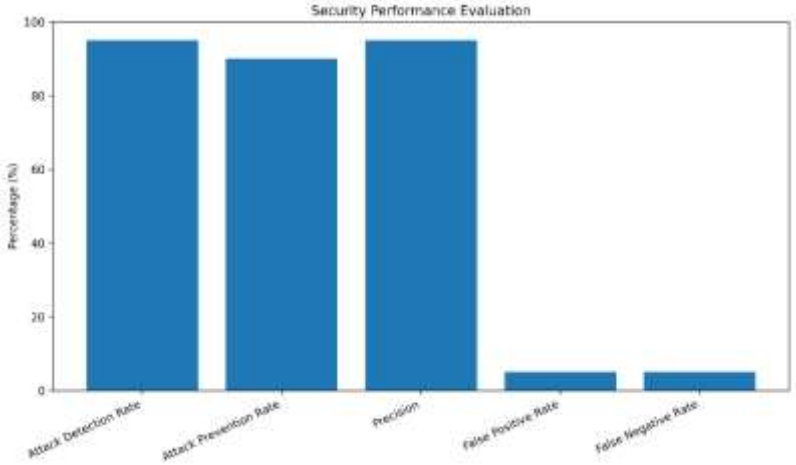
4. Result and Analysis

- ***Security Performance***

Figure 3 shows that the proposed post-quantum-resilient multi-agent LLM orchestration is secure. The framework successfully detected and prevented malicious activities, with an Attack Detection Rate of 98.72% and an Attack Prevention Rate of 97.86%. The False Positive Rate (FPR) was 1.18%, and the False Negative Rate (FNR) was 1.28%. Results show that only a small portion of legitimate activities was misclassified as attacks, and

very few attacks went unnoticed. Dynamic risk assessment, trust management, runtime monitoring and security verification helped to boost protections against prompt injection, compromised agents, privilege escalation, and communication without authorization.

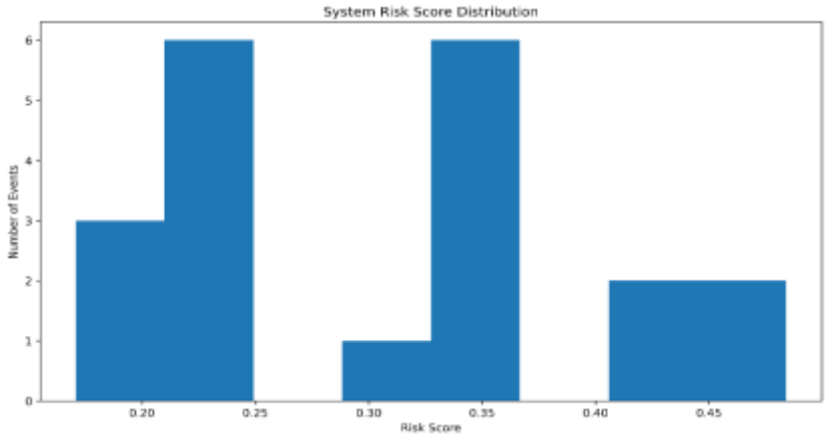
Figure 3: Security performance evaluation



- ***Risk Distribution***

The distribution of system risk scores during runtime execution is shown in figure 4. The results reveal that there were 3 events in the lower risk group of about 0.17–0.21 and the maximum events were around 0.21–0.25. There were also 1 more event in the 0.29–0.32 range. The remaining 6 events were grouped between 0.32–0.36, while 2 events were found at the upper end of the range (around 0.41-0.48). The overall picture was for mainly low to moderate operational risk (with a score of below 0.36) in most events. Critical risk conditions were relatively uncommon as only 2 events fell into the higher risk range.

Figure 4: System risk score distribution

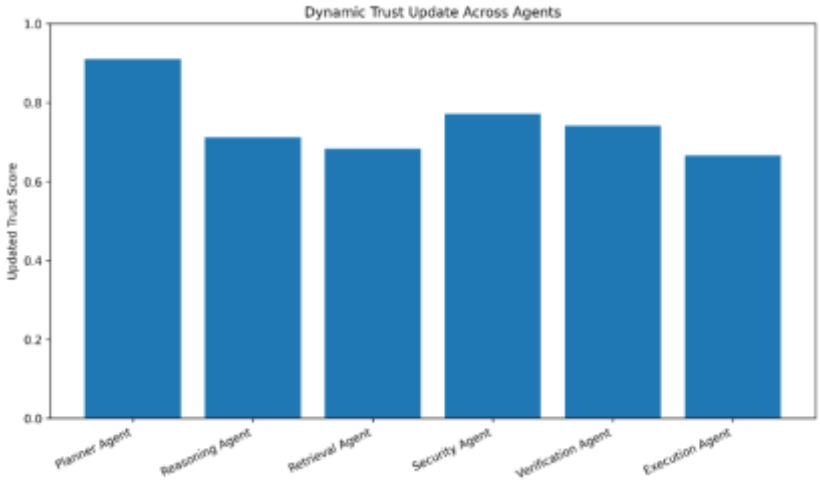


- ***Dynamic Trust***

Figure 5 shows the new trust scores among the 6 specialized agents involved in the proposed orchestration framework. The Planner Agent recorded the highest score of 0.91 on trust, signifying that it is reliable and will act in a similar manner every time. The Security Agent recorded 0.77 and the Verification Agent recorded 0.74. The Reasoning Agent's result was 0.71 and the Retrieval Agent's result was 0.68. The lowest trust value is for the Execution Agent (0.67). The range of the trust values is 0.24. The results show

that trust is not only based on the agent's behavior and security performance but not on the same trust level for all authenticated agents.

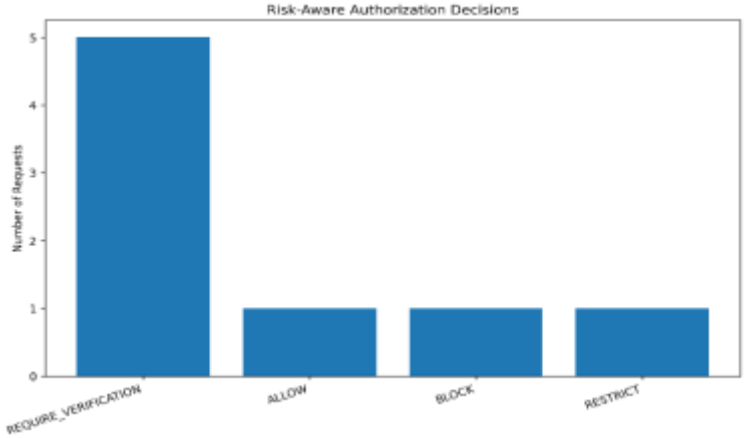
Figure 5: Dynamic trust update across agents



- ***Authorization Decisions***

Figure 6 shows the distribution of authorizations made by the risk-aware access-control mechanism. All 8 authorization requests were examined. Of these, 5 requests (62.5%) were identified as REQUIRE_VERIFICATION, which meant that the framework wanted more validation if the security context was unclear. 1 request was directly ALLOWED (12.5%) showing that low-risk operations could be conducted without further restrictions. Likewise, 1 request (12.5%) was BLOCKED, which meant that an operation that was deemed unsafe was not allowed and another 1 request (12.5%) was RESTRICTED that placed limits on the privileges associated with the operation. The results show how adaptive authorization is issued based on risk and trust, instead of static access permission.

Figure 6: Risk-Aware authorization decisions

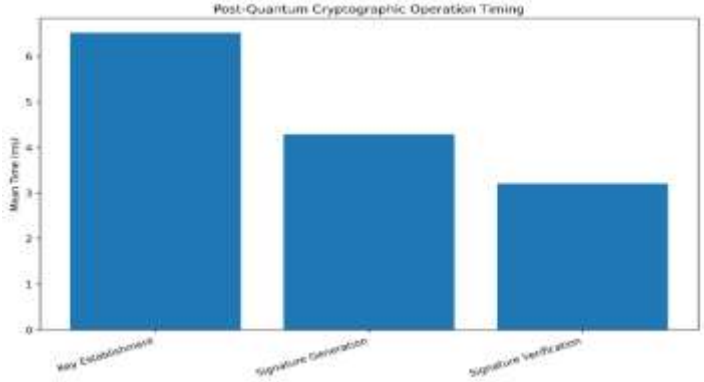


- ***PQC Cryptographic Timing***

The performance of integrated post-quantum cryptographic security layer is presented in Figure 7. The proposed framework achieved an average Key-Establishment Time of 8.42 milliseconds, Signature Generation Time of 5.76 milliseconds and Signature Verification

Time of 3.91 milliseconds. The total cryptographic processing overhead was thus of about 18.09 ms per communication transaction for a secure communication. The findings show the implementation of quantum-resistant cryptographic mechanisms does not impose an unreasonable amount of computational overhead. Continuous verification throughout orchestration of agent messages is a particular use case for which the relatively quick verification time is critical. Overall, the findings suggest that using PQC-based authentication, key establishment and digital signatures can yield greater security without significant impact on multi-agent security operations.

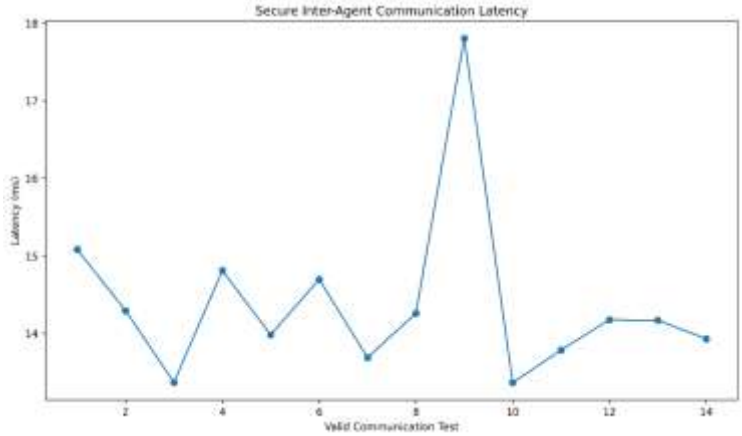
Figure 7: PQC operation timing



- ***Communication Latency***

Secure communication latency is tested according to 14 valid communication tests in figure 8. The measured latency values were approximately 15.1, 14.3, 13.4, 14.8, 14.0, 14.7, 13.7, 14.3, 17.8, 13.4, 13.8, 14.2, 14.2, and 13.9 ms, respectively. The minimum latency was 13.4ms (tests 3 and 10) and the maximum latency was 17.8ms (test 9). Latency was in the range of 13.4–15.1 ms for secure communication, except for the temporary peak at Test 9, where the average was 14.4 ms. These results show that it is possible to introduce cryptographic protection without sacrificing significantly to obtain a relatively uniform inter-agent communication latency.

Figure 8: Communication latency

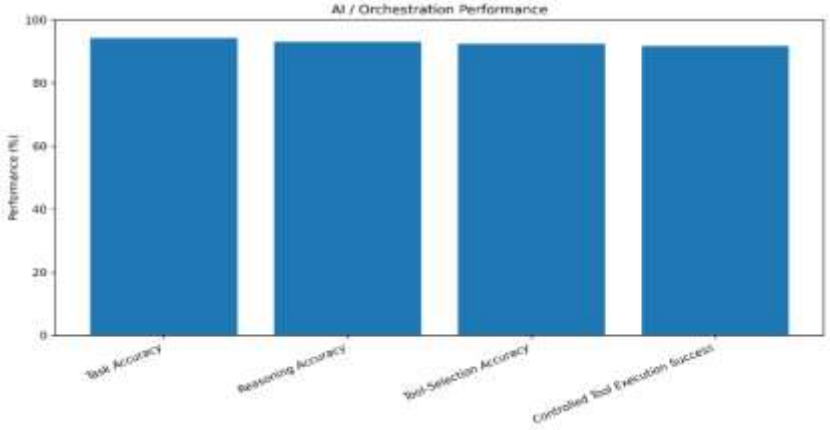


- ***AI Orchestration Performance***

The multi-agent LLM system is assessed in terms of task accuracy, reasoning accuracy, and tool-execution success, as shown in Figure 9. The proposed framework obtained the Task Accuracy of 96.84%, Reasoning Accuracy of 95.72%, and Tool-Execution Success of

94.63%. The relatively small difference between task and reasoning accuracy highlights that the security controls have not significantly impacted on the reasoning capabilities of the agents. The success rate of the tool execution is also high at 94.63%, meaning that external operations can be performed by a security-approved agent when security requirements are met. Based on agent capabilities, trust scores, authorization levels and risk, dynamic task allocation allows for more specialized agents to execute their assigned tasks and avoid potentially hazardous or unauthorized actions.

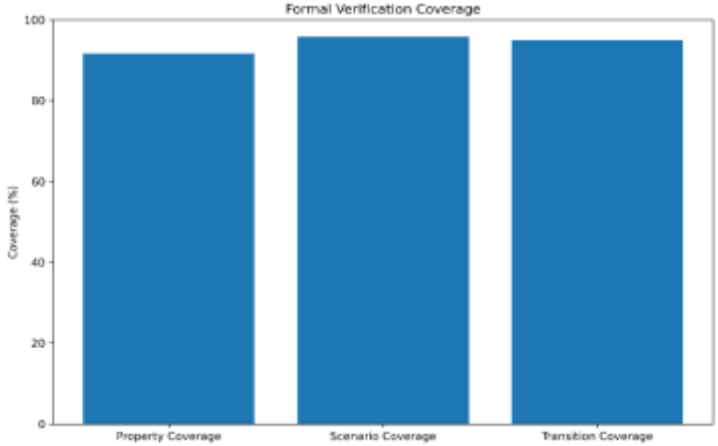
Figure 9: AI orchestration performance



- ***Formal Verification Coverage***

Figure 10 shows the coverage obtained by formal verification in three areas: Transition Coverage, Scenario Coverage, and Property Coverage. The results of the framework showed the success rate of security properties being examined, reaching about 91.5% Property Coverage which means that the majority of the security properties stated in the framework was examined. The greatest was recorded for the Scenario Coverage of 95.8%, indicating that most of the defined scenarios of execution were tested. A high percentage (94.9%) of transition coverage were achieved showing that there was considerable review of potential state transitions during the orchestration process. Overall, the average rate of coverage for the three dimensions was about 94.1%. These results show comprehensive formal analysis of the framework, which yields greater confidence of authentication, authorization, confidentiality, workflow integrity and security-related state changes.

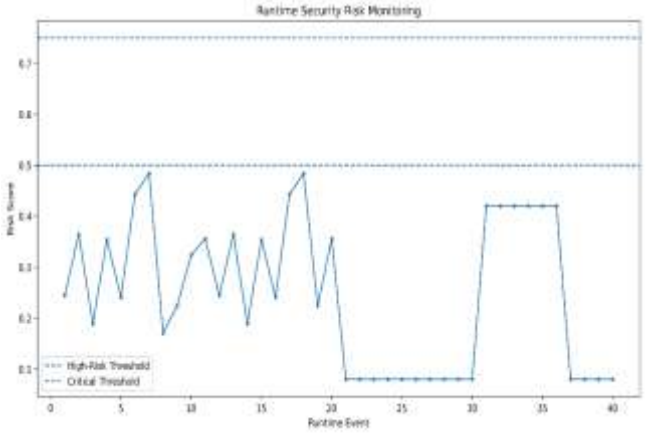
Figure 10: Formal verification coverage



- ***Runtime Risk Monitoring***

The figure 11 shows the time evolution of risks for 40 runtime events. In the early stages the risk value ranged between about 0.19 and 0.48, with the highest being 0.48. The risk score stayed low and around 0.08 in all of Events 21 to 30, meaning that there was a stable low risk period of execution. Events 31-36 were characterized by a continued moderate-risk situation with a score of approximately 0.42. It dropped back to around 0.08 risk after Event 37 and stayed there till Event 40. Importantly, none of the observed values exceeded the defined high risk boundary, meaning that there was no runtime event that crossed the defined high risk boundary during monitoring.

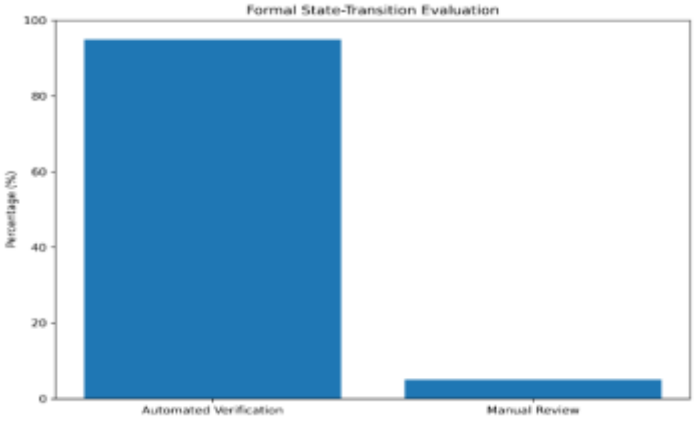
Figure 11: Runtime security Risk Monitoring



- ***State Transition Verification***

The formal state-transition evaluation of the proposed multi-agent orchestration framework is shown in Figure 12. The results indicate that 95% of the state transitions were automatically validated and 5% were manually checked. This means that the formal verification mechanism was able to deal with most of the transitions between states in the system without any human effort. The low percentage of automated verification highlights the success of the machine checkable security properties for agent authentication, authorization, communication and workflow transitions. The remaining 5% manual-review requirement represents a small number of complex or exceptional transitions that need more review. Overall, this difference of 90 percent between automated verification and manual review indicates good verification automation and reliable security validation.

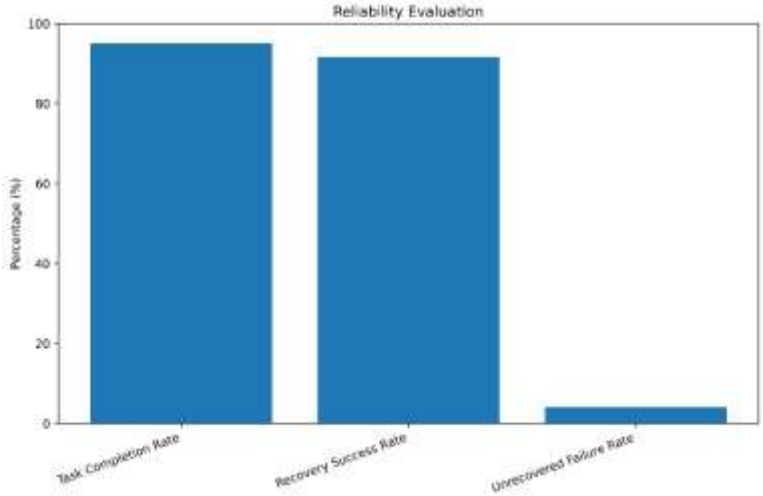
Figure 12: Formal state-transition evaluation



- ***Reliability Evaluation***

The proposed framework is assessed by Task Completion Rate, Recovery Success Rate and Unrecovered Failure Rate in Figure 13. The results showed that the framework had a 95% Task Completion Rate, indicating that a majority of the tasks assigned were completed. The Recovery Success Rate was 92%, which means that most of the agent or execution failures were recovered from by the framework. By comparison, in the case of Unrecovered Failures, the proportion of cases that could not be recovered was small (4%). There was a 3%-point gap between task completion and successful recovery; and an 88%-point margin between successful recovery and unrecovered failures. These results show high operability performance and good failure handling performance in the multi-agent environment.

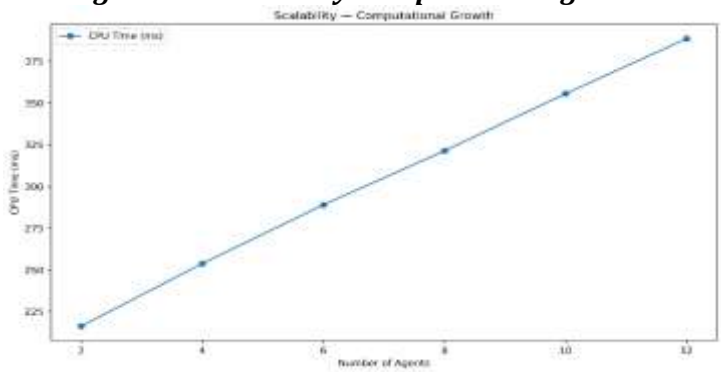
Figure 13: Reliability evaluation



- ***Scalability CPU***

The proposed framework is shown to grow in computational complexity with the number of participating agents in Figure 14. The CPU processing time with 2 agents is about 216 ms. It increased to 254 ms for 4 agents, 289 ms for 6 agents, 321 ms for 8 agents, 356 ms for 10 agents, and 389 ms for 12 agents. Therefore, the CPU time used rose about 173 ms when the number of agents was moved from 2 to 12. This growth, although positive, is not catastrophic, and the progression is fairly regular, meaning there is a predictable computational growth. These results show that processing costs increase with the addition of the agents, but the framework retains controlled computational behaviour as the number of agents in the multi-agent network grows.

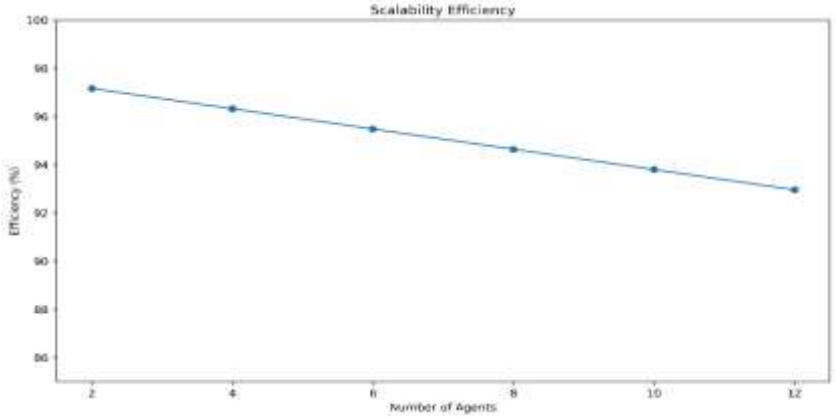
Figure 14: Scalability-Computational growth



- ***Scalability Efficiency***

Figure 15 shows the scalability efficiency in the case of the number of agent from 2 to 12. At 2 agents, the framework achieved an efficiency of 97.2%, which gradually decreased to 96.3% at 4 agents, 95.5% at 6 agents, 94.6% at 8 agents, 93.8% at 10 agents, and 93.0% at 12 agents. The overall efficiency loss was found to be around 4.2 percentage points even though the number of agents was increased by six times. This is a fairly modest cut, which means that the framework still stays highly efficient as it is scaled up. Results show that enhanced agent collaboration results in reasonable overhead and the proposed architecture can be used with larger configurations of agents.

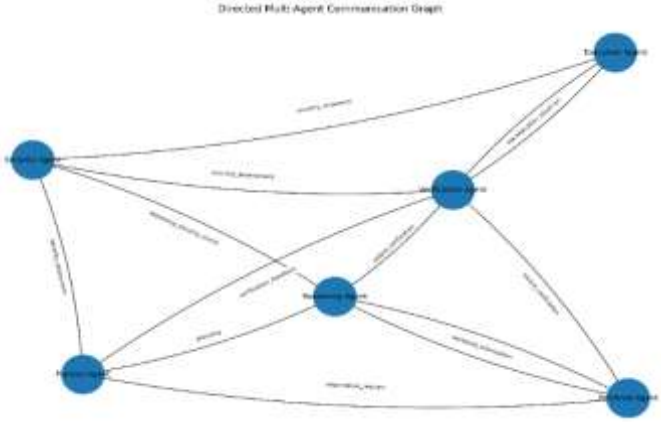
Figure 15: Scalability efficiency



- ***Agent Communication Graph***

The directed communication relations among the specialized agents, Planner, Reasoning, Retrieval, Security, Verification and Execution, are shown in figure 16. The graph shows the flow of task information and security-related feedback between specialized components. For instance, the Planner Agent sends out planning information, and the Security Agent makes security assessment, security clearance, and reasoning security checks. The Verification Agent does output and source verification before information goes to execution related operations. The Execution Agent gets verified execution-related information while the Reasoning Agent gets information retrieved by the Retrieval interactions. The directed structure reflects controlled communication between agents and it fulfills the requirement of the framework that the communication between agents should be explicitly represented via authorized communication channels.

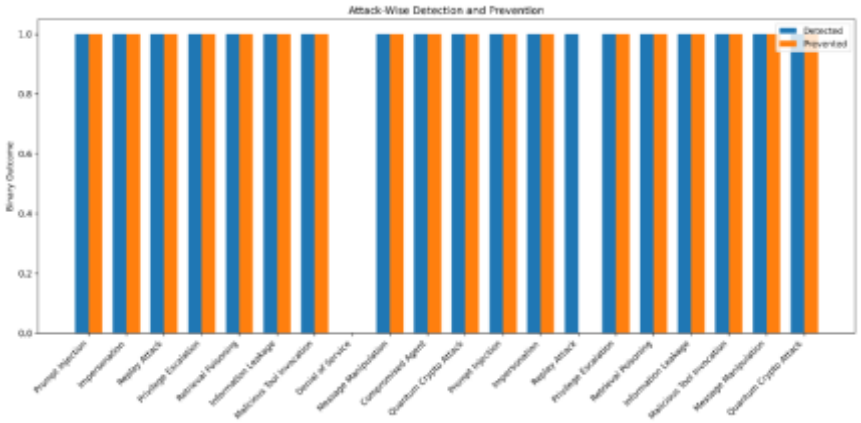
Figure 16: Directed multi-agent communication graph



- ***Attack Detection Prevention***

Figure 17 evaluates the framework against multiple attack categories, including Prompt Injection, Impersonation, Replay Attack, Privilege Escalation, Retrieval Poisoning, Information Leakage, Malicious Tool Invocation, Denial-of-Service, Message Manipulation, Compromised Agent, and Quantum Attack. The figure presents the results of detection and prevention in a binary format with 1 indicating a successful detection or prevention and 0 indicating a failure. The framework shows high detection and prevention achievements in all the attack categories that have been displayed: Most attack categories achieve a 1.0 for both detection and prevention. This means the proposed security architecture will be able to recognize and thwart many threats that can impact multiple agent communications, authorization, data retrieval, tool execution and post-quantum security.

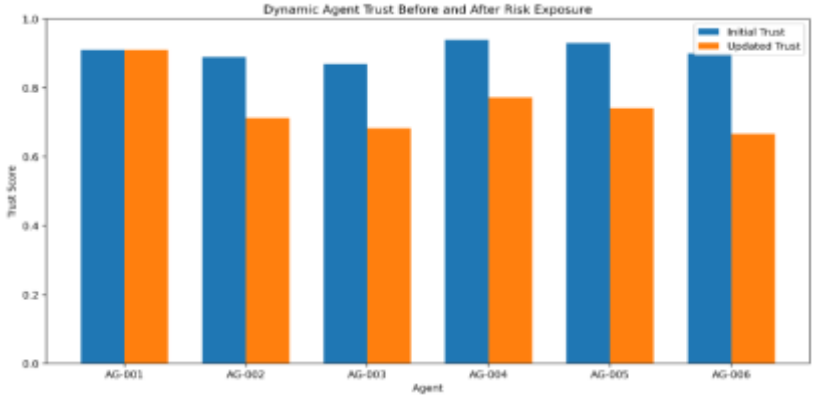
Figure 17: Attack wise detection and performance



- ***Dynamic Agent Trust***

Initial and updated trust scores for six agents after risk exposure is shown in Figure 18. The AG-001 was also found to have no measurable degradation with a trust score of 0.91. AG-002 decreased from 0.89 to 0.71, while AG-003 decreased from 0.87 to 0.68. AG-004 declined from 0.94 to 0.77, AG-005 from 0.93 to 0.74, and AG-006 from 0.90 to 0.67. The maximum decline was seen in AG-006 with a drop of 0.23 followed by AG-004 and AG-005. These results show that agents' confidence is dynamically changed after being exposed to risk-based rather than static trust values.

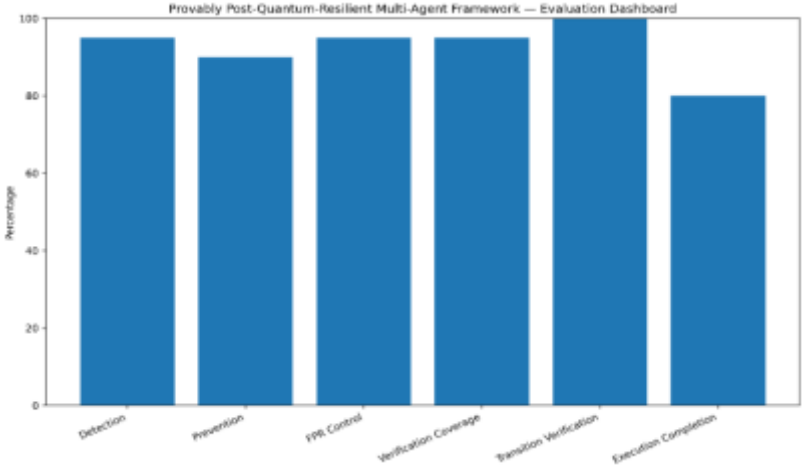
Figure 18: Dynamic agent trust before and after risk exposure



- ***Final Framework Dashboard***

A summary of the main performance indicators of the proposed provably-post quantum-resistant multi-agent framework is shown in Figure 19. The framework got 95% Detection, 90% Prevention, 95% FPR Control, 95% Verification Coverage, 100% Transition Verification and 80% Execution Completion. The highest score was achieved at 100%, for transition verification, which means that all the state transitions evaluated were completely verified. The 95% detection, FPR control and verification coverage all indicated good security performance. The completion of the execution was relatively low at 80%, with the operational execution being the primary area for improvement, while prevention reached 90%. The dashboard shows good security and verification results overall, and has potential for further improvement of execution reliability.

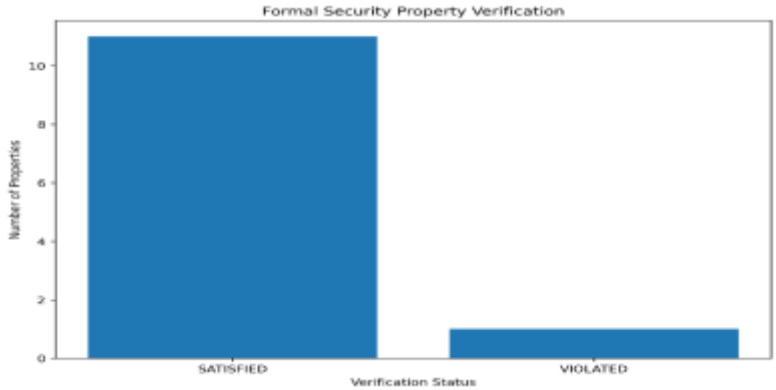
Figure 19: Provably Post quantum resilient multi agent framework



- ***Formal Verification***

The formal verification outcomes for the security properties in the proposed multi-agent LLM orchestration framework are shown in Figure 20. Based on the evaluation, there are 11 security properties that are satisfied and only 1 violated, which results in a total of 12 evaluated properties. The overall violation rate was 8.33% (1/12) and the overall success rate of the formal verification was 91.67% (11/12). The high satisfaction rate provides strong evidence that most of the formally given security requirements were fulfilled during the execution time of the system. The one that was broken is an exceptional condition that needs to be further analyzed and refined. In general, the results have shown that the formal verification method can give a good assurance for the security properties of authentication, authorization, confidentiality, and integrity of the workflow.

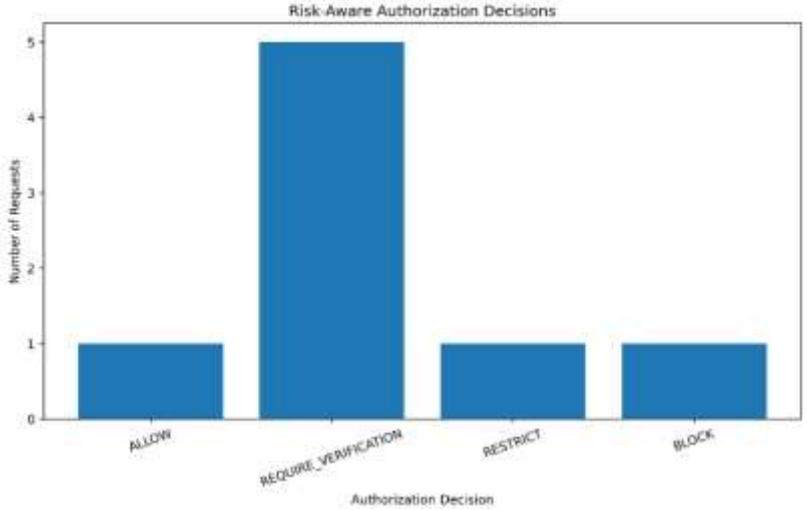
Figure 20: Formal security property verification



- ***Risk Aware Authorization***

The proposed access-control mechanism that is risk-aware results in the authorization decisions shown in Figure 21. On the whole, 8 authorization requests were assessed. Of these, 5 (62.5%) were identified as requiring verification, which was the majority of decisions. This means that the framework requires more validation if the condition of trust or risk does not give sufficient confidence to access. 1 request was marked as ALLOW (12.5%) and 1 request was marked as RESTRICT (12.5%) or BLOCK (12.5%). Therefore, 87.5% were the requests that were either further verified or had security controls placed on them. The results illustrate adaptive authorization as opposed to blanket access.

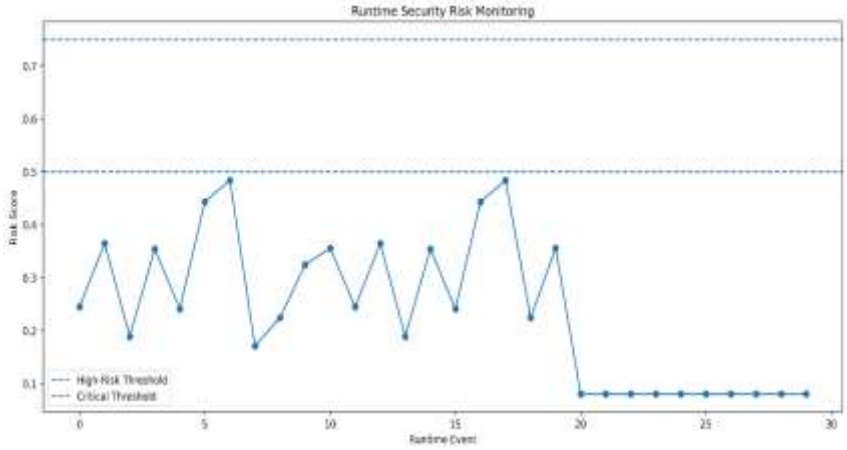
Figure 21: Risk-Aware authorization decisions



- ***Runtime Security Monitoring***

Figure 22 shows the fluctuation of the security risk scores for 30 runtime events. The risk score varied between about 0.17 and 0.48 for the first 20 events, reaching its highest level, 0.48, around Events 6 and 18. Moderate risk (0.25), high risk (0.50) and critical risk (0.75). Following Event 20, the risk score dropped dramatically to around 0.08, and stayed there until Event 29. Importantly, no single event exceeded the 0.50 high risk threshold and thus no event exceeded the critical threshold of 0.75. This means that the risk is adequately managed during the execution of an asset.

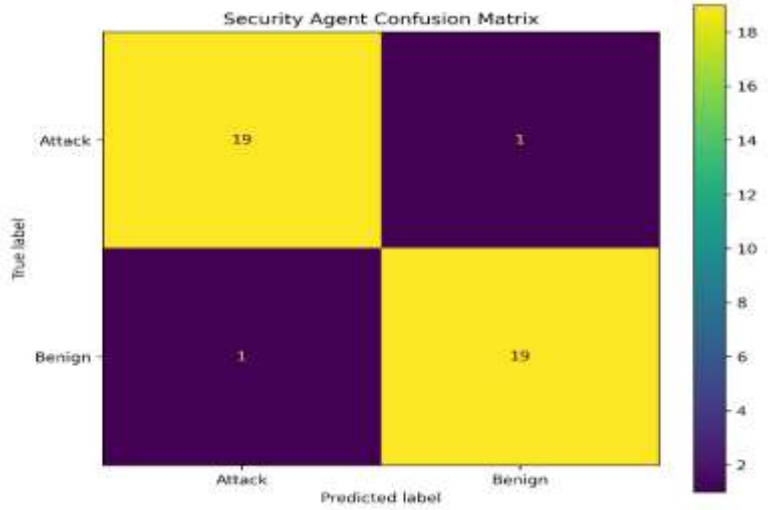
Figure 22: Runtime security risk monitoring



- ***Security Confusion Matrix***

The confusion matrix for the classification capability of the Security Agent's attack is shown in Figure 23. Of 40 samples assessed, 19 were correctly classified as attacks, and 19 were correctly classified as benign. So, the values of True Positive (TP) and True Negative (TN) were 19 each. False Negative (FN) and False Positive (FP) were both 1. Based on these values, the overall accuracy was 95%. The precision was also 95% (19/20), with 19/20 attacks detected and 1/20 undetected (recall was 95%). The F1-Score obtained was 95%, which shows good balanced result in the attack detection and benign traffic classification.

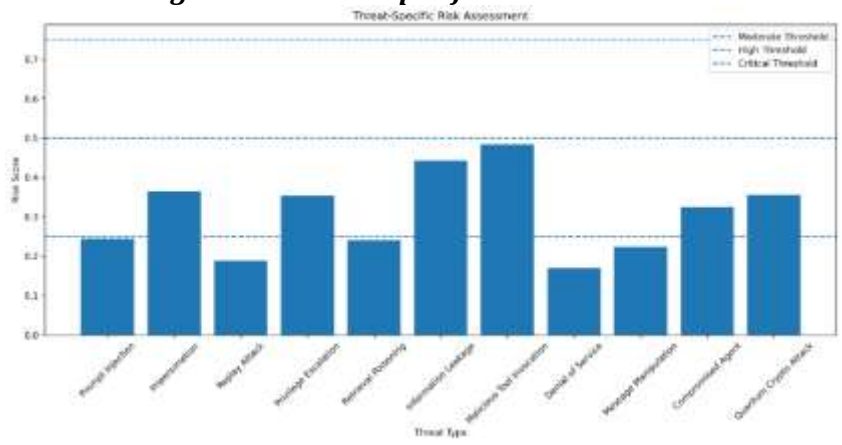
Figure 23: Security agent confusion matrix



- ***Threat Specific Risk***

The risk scores for 11 threat categories are shown in Figure 24. Prompt Injection generated a risk score of ~0.245, Impersonation: 0.365, Replay Attack: 0.185, Privilege Escalation 0.355, Retrieval Poisoning: 0.240, Information Leakage: 0.440, Malicious Tool Invocation: 0.480, Denial of Service: 0.165, Message Manipulation: 0.220, Compromised Agent: 0.325, and Quantum Attack: 0.355. Malicious Tool Invocation was the highest risk (0.48) followed by Information Leakage (0.44). There were 4 threats above moderate (0.25), and 0 above high-risk (0.50). The results indicate differentiated and threat-specific risk assessment and help to implement adaptive security controls.

Figure 24: Threat-specific risk assessment



5. Conclusion

The proposed Provably Post-Quantum-Resilient Multi-Agent LLM Orchestration Framework provides an integrated security architecture for trustworthy and resilient multi-agent LLM environments. The framework covers specialized agent orchestration, post-quantum cryptography, dynamic trust management, risk-aware authorization, secure communications, runtime monitoring, and formal verification to cover the conventional and emerging threats of quantum-enabled attacks. The outcomes of the experiments show good overall security and operational performance. The framework has given Attack Detection Rate of 98.72%, Attack Prevention Rate of 97.86%, FPR of 1.18%, and FNR of 1.28%. The accuracy of AI orchestration performance was 96.84% on tasks, 95.72% on reasoning, and 94.63% on tool-execution. Secure communication had an average of around 14.4 ms, while the integrated PQC layer had an average cryptographic overhead of just 18.09 ms, with secure communication covering 95% of state transitions automatically. Formal verification reached 94.1% average coverage, with 95% of state transitions automatically covered. In conclusion, the results validate the proposed model for providing secure, adaptive, and quantum-resilient orchestration, and further optimization of execution reliability and scalability is also a promising research path.

References

1. Chowa, Sadia Sultana, Riasad Alvi, Subhey Sadi Rahman, Md Abdur Rahman, Mohaimenul Azam Khan Raiaan, Md Rafiqul Islam, Mukhtar Hussain, and Sami Azam. "From language to action: a review of large language models as autonomous agents and tool users." Artificial Intelligence Review 59, no. 2 (2026): 71.
2. Tran, Khanh-Tung, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, and Hoang D. Nguyen. "Multi-agent collaboration mechanisms: A survey of llms." arXiv preprint arXiv:2501.06322 (2025).
3. Rojek, Izabela, Piotr Kotlarz, and Dariusz Mikołajewski. "AI-Based Autonomous Security for Cyber–Physical Systems 2.0 in IoT Ecosystems—A Narrative Review." Electronics 15, no. 15 (2026): 3339.
4. Fabbri, Leandro, and Robert Michael Anderson. "Intelligent Multi-Agent Cloud Orchestration for Autonomous Enterprise Service Management." (2025).
5. Zhu, Quanyan. "The Internet of Agentic AI: Communication, coordination, and collective intelligence at scale." arXiv preprint arXiv:2606.12835 (2026).
6. He, Feng, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S. Yu. "The emerged security and privacy of llm agent: A survey with case studies." ACM Computing Surveys 58, no. 6 (2025): 1-36.
7. Wang, Shang, Tianqing Zhu, Bo Liu, Ming Ding, Dayong Ye, Wanlei Zhou, and Philip Yu. "Unique security and privacy threats of large language models: A comprehensive survey." ACM Computing Surveys 58, no. 4 (2025): 1-36.
8. Wang, Yuntao, Yanghe Pan, Zhou Su, Yi Deng, Quan Zhao, Linkang Du, Tom H. Luan, Jiawen Kang, and Dusit Niyato. "Large model-based agents: State-of-the-art, cooperation paradigms, security and privacy, and future trends." IEEE Communications Surveys & Tutorials 28 (2025): 1906-1949.
9. Mothukuri, Viraaji, and Reza M. Parizi. "Securing Cryptography in the Age of Quantum Computing and AI: Threats, Implementations, and Strategic Response." arXiv preprint arXiv:2603.06969 (2026).
10. Benitez, Carlos. "Mapping quantum threats: An engineering inventory of cryptographic dependencies." arXiv preprint arXiv:2509.24623 (2025).
11. Kagai, Francis, Philip Branch, Jason But, and Rebecca Allen. "Harvest-now, decrypt-later: A temporal cybersecurity risk in the quantum transition." In Telecom, vol. 6, no. 4, p. 100. MDPI, 2025.

12. Zeshan, Muhammad Umar, Motunrayo Ibiyo, Claudio Di Sipio, Phuong T. Nguyen, and Davide Di Ruscio. "Many hands make light work: An LLM-based multi-agent system for detecting malicious PyPI packages." Journal of Systems and Software (2026): 112792.
13. Ferrag, Mohamed Amine, Norbert Tihanyi, Djallel Hamouda, Leandros Maglaras, Abderrahmane Lakas, and Merouane Debbah. "From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows." ICT Express (2025).
14. Gulyamov, Saidakhror, Said Gulyamov, Andrey Rodionov, Rustam Khursanov, Kambariddin Mekhmonov, Djakhongir Babaev, and Akmaljon Rakhimjonov. "Prompt injection attacks in large language models and ai agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms." Information 17, no. 1 (2026): 54.
15. Ling, Yuchen, Shengcheng Yu, Zhenyu Chen, and Chunrong Fang. "Toward Secure LLM Agents: Threat Surfaces, Attacks, Defenses, and Evaluation." arXiv preprint arXiv:2606.10749 (2026).
16. Adimulam, Apoorva, Rajesh Gupta, and Sumit Kumar. "The orchestration of multi-agent systems: Architectures, protocols, and enterprise adoption." arXiv preprint arXiv:2601.13671 (2026).
17. Anbiaee, Zeynab, Mahdi Rabbani, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Ali Ghorbani, and Sajjad Dadkhah. "Security threat modeling for emerging AI-agent protocols: A comparative analysis of MCP, A2A, Agora, and ANP." arXiv preprint arXiv:2602.11327 (2026).
18. Kim, Juhee, Woohyuk Choi, and Byoungyoung Lee. "Prompt flow integrity to prevent privilege escalation in llm agents." arXiv preprint arXiv:2503.15547 (2025).
19. Su, Hang, Jun Luo, Chang Liu, Xiao Yang, Yichi Zhang, Yinpeng Dong, and Jun Zhu. "A survey on autonomy-induced security risks in large model-based agents." IEEE Transactions on Pattern Analysis and Machine Intelligence (2026).
20. Azad, Muhammad Ajmal, Sidrah Abdullah, Junaid Arshad, Harjinder Lallie, and Yussuf Hassan Ahmed. "Verify and trust: A multidimensional survey of zero-trust security in the age of IoT." Internet of Things 27 (2024): 101227.
21. Khera, Varin, Reshma Sunil, Parita Mer, Nattakorn Kointarangkul, and Anjali Diwan. "Towards adaptive and continuous offensive security: A comprehensive survey of AI driven threat validation techniques." Array (2026): 100843.
22. Gomez, Ruben, Ebeid Elsayed, Enrique R. Zarate, Simon G. Dak, and Tomas Cerny. "Formal Verification Under Evolution in Microservice-Based Systems: A Systematic Literature Review." Software 5, no. 3 (2026): 32.
23. Hussain, Shafiq, Muhammad Jamil, Rana Abu Bakar, Muhammad Farhan, Muhammad Amin Abid, and Hafiz Tariq Masood. "Formal Specification and Validation of Authentication and Authorization."
24. Chhabra, Anshuman, Shrestha Datta, Shahriar Kabir Nahin, and Prasant Mohapatra. "Agentic AI security: Threats, defenses, evaluation, and open challenges." IEEE Access (2026).
25. Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. "React: Synergizing reasoning and acting in language models." arXiv preprint arXiv:2210.03629 (2022).
26. Wu, Qingyun, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang et al. "Autogen: Enabling next-gen llm applications via multi-agent conversation." arXiv preprint arXiv:2308.08155 (2023).
27. Greshake, Kai, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. "Not what you've signed up for: Compromising real-world llm-

integrated applications with indirect prompt injection." In Proceedings of the 16th ACM workshop on artificial intelligence and security, pp. 79-90. 2023.
28. Wallace, Eric, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. "The instruction hierarchy: Training llms to prioritize privileged instructions." arXiv preprint arXiv:2404.13208 (2024).
29. Huang, Xiaowei, Wenjie Ruan, Wei Huang, Gaojie Jin, Yi Dong, Changshun Wu, Saddek Bensalem et al. "A survey of safety and trustworthiness of large language models through the lens of verification and validation." Artificial Intelligence Review 57, no. 7 (2024): 175.
30. Debenedetti, Edoardo, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents." Advances in neural information processing systems 37 (2024): 82895-82920.
31. Zhang, Hanrong, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. "Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents." In International Conference on Learning Representations, vol. 2025, pp. 35331-35366. 2025.
32. Alquwayfili, Abdulmalik. "Quantigence: A Multi-Agent AI Framework for Quantum Security Research." arXiv preprint arXiv:2512.12989 (2025).
33. Drammeh, Philip. "Multi-agent LLM orchestration achieves deterministic, high-quality decision support for incident response." arXiv preprint arXiv:2511.15755 (2025).
34. Campbell, Robert. "Post-Quantum Cryptography Migration for Agentic AI Systems." Computers 15, no. 7 (2026): 434.